

as a basis for generating the final response. This process of retrieve-refine-generate, as shown in the architectural flow diagram (Figure 1), provides assurance that the LLM's answer is built on externally verified source material rather than solely on its internally trained data.

There are two pipelines in RAG architecture: offline and online. Offline pipelines create knowledge before the user sends queries by reading source documents, cleaning and standardising text, cutting content into pieces with clear meanings, adding more information about the item, and creating and storing embeddings of the item in a vector database. This type of processing is done with an emphasis on the rate of processing (throughput), not the speed at which it is processed. Therefore, offline processing can happen using a batch.

Online pipelines respond to the request of an end user in real-time and have strict guidelines regarding latency. The online pipeline includes query rewriting, retrieval of related items, re-ranking the retrieved items, constructing the context, and running inference with an LLM.

By separating the offline and online pipelines, the overall system can scale without affecting the performance of real-time queries due to large and costly indexing.

How does a production-grade RAG architecture work?

Production-ready RAG architectures are heavily influenced by the latency and cost constraints presented on them. As an example, every stage of your online pipeline contributes to your overall latency budget, and LLM inference is typically the most latency- and cost-contributing stage. Given that LLM inference is often the most expensive stage in your production-ready RAG pipeline, retrieval and re-ranking must both be fast and efficient, ensuring that a limited number of high-quality chunks are fed into your model.

In addition, from a cost perspective, architecture must be designed such that system resources are progressively used. This means that retrieval operations (which are typically low-cost) must be performed prior to higher-cost operations like re-ranking and LLM generation, in order to identify the context needed to perform them. This layer-oriented design approach ensures predictable performance, controlled token usage, and the ability to scale as both data volume and query traffic increase.

Data ingestion: Preparing knowledge for retrieval

The process of data ingestion is a crucial element of a production-ready RAG system — it changes raw enterprise data into knowledge that can be retrieved. In the production environment, ingestion is an ongoing process whereby an automated pipeline is created to maintain data quality, currency, and traceability.

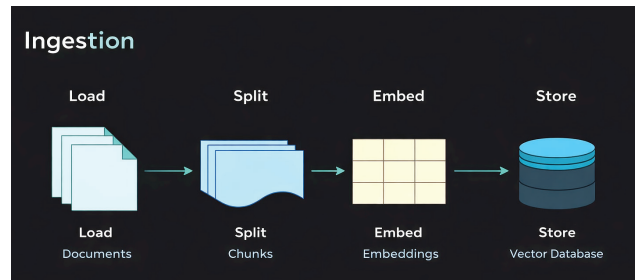


Figure 2: Data ingestion

Step 1. Source types (Load): RAG systems can ingest data from several different sources (e.g., PDFs, HTML pages, Markdown files, databases, APIs, knowledge bases, and other internal systems). Each type of source requires a specific type of loader to extract the content properly and maintain the document boundaries, timestamps, and access control, all of which are necessary for secure and auditable retrieval.

Step 2. Cleaning and normalisation (Split): Once loaded into the RAG system, documents undergo a cleaning process to remove noise such as headers, footers, navigation elements, and duplicate content. During this process, the text is normalised (ensuring that all the text uses the same encoding and formatting) and documents are split into semantically meaningful size chunks. This ensures that low-quality text is not embedded in the RAG system, which would then degrade the precision of subsequent retrievals.

Step 3. Metadata enrichment (Embed): Each chunk of data is enriched with structured metadata (i.e., source, document type, language, version, and date of update). The metadata enables filtered, hybrid and context-enabled retrievals in production systems. The chunks of data are then processed to generate dense vector embeddings of the data. The processing of the chunks is typically performed using a batched approach, which allows for lower cost and higher throughput.

Step 4. Update handling (Store): Storing all embeddings and their associated metadata occurs through a vector database. Incremental updates occur in production environments by only re-embedding any new or changed documents for production use and by deprecating old content with versioning and soft deletes. By using these processes for deprecation of old or outdated content, the accuracy of retrieval from the vector database will not require the entire corpus to be reprocessed.

Chunking and the types of chunking strategies

The process of breaking down raw documents into smaller units of knowledge for use in embedding and retrieval processes is called chunking. The design of the chunks directly impacts the retrieval accuracy, context relevance, and ability to control hallucination in RAG systems. Poorly